BrandGuard AI

An Evidence-Based Brand Governance and Content Review Platform

Round 2: The Build

Problem statement: Inkloom AI, Intelligent Brand Governance Platform

Team: HACKSMITH

Table 1. Where to find the working prototype.

Item	Link or value
Live prototype	https://hacksmith.vercel.app/login
Source repository	https://github.com/sagarkharbikar25/HACKSMITH- (public)
Version	Main branch of the public repository, as it stood at submission.
API health and docs	https://hacksmith.onrender.com/api/health/
Demo login	Creator: saar@test.com, password 12345678 Reviewer: ss@test.com, password 12345678 Brand Manager: sk@test.com, password 12345678

Status of this document. This document describes the Round 2 build as it runs at the live link. Every feature in Section 4 carries a status. Section 10 lists the scenarios we tested by hand. We have not measured precision or recall yet, so we report no accuracy figures. Known problems are listed in Section 15.

Contents

1. Executive Summary....3
2. Round 2 Submission at a Glance....4
3. Problem and Round 1 Recap....5
4. Completed Features....5
5. Core Workflow in the Prototype....7
6. Architecture and Technology Stack....10
7. Audit Engine and AI Methodology....11
8. Data Model and API Contract....13
9. Security, Privacy and Responsible Design....**Error! Bookmark not defined.**
10. Testing and Evaluation Results....14
11. Deployment and Reliability....16
12. Changes from the Round 1 Plan....16
13. Team Contributions and Execution....17
14. Three Major Round 3 Improvements....18
15. Limitations, Risks and Known Issues....19
16. Conclusion....20
References....21
Appendix A. Sample Knowledge Base and Audit Response....22
Appendix B. Alignment with Round 2 Judging Criteria....22
Tools and AI Disclosure....23

1. Executive Summary

BrandGuard AI is a pre-publication brand governance and content review platform, built for the Inkloom AI "Intelligent Brand Governance Platform" problem statement [1]. Round 1 was a plan. In Round 2 we built it and deployed it at https://hacksmith.vercel.app/login, so a judge can open the link and run the core workflow without any setup.

The workflow runs from start to finish. A creator writes copy in Creator Studio and runs an audit. A deterministic rule engine checks the text for superlatives, territory claims that contradict approved product facts, and banned hype terms. Each finding comes back with exact character offsets, so the highlights land on the right words. One click applies a suggested fix, re-audits the text and saves it as Version 2. A draft that still has open critical or high-severity findings cannot be submitted. A clean draft goes to the Review Terminal, where a reviewer sees Version 1 and Version 2 side by side and records a decision. Approving a draft that still carries warnings needs a written justification. Every creation, audit and approval goes into a tamper-evident audit trail that never stores the raw copy, and the Brand Governance Center shows brand managers which rules fire most and which get dismissed.

Findings fall into four types: `confirmed_contradiction`, `unsupported_claim`, `style_issue` and `needs_human_verification`. The type tells the reviewer how strong the evidence is, and a missing approved source is never reported as "this is false". Only the deterministic engine can raise a `confirmed_contradiction`, because a contradiction needs a stored approved fact. Gemini 1.5 Flash adds advisory style and tone suggestions without blocking the rule engine. If it is unavailable, the status bar says so and the rule-based audit carries on alone.

Round 2 also forced decisions the plan did not cover. We moved production data to PostgreSQL because free host disks are ephemeral. We merged the separate legal and marketing review screens into one Compliance Reviewer role. We turned the advisory warning into a hard gate that blocks submission. We also added automatic redaction of confidential facts for agency accounts. Section 12 lists every change from Round 1 and the reason for it.

At submission, 14 of the 16 planned Round 2 features are complete (Section 4). Two are partial. Section 10 lists the scenarios we tested and Section 15 lists what does not work yet. Section 14 describes the three Round 3 improvements: guideline ingestion from documents, visual and accessibility compliance, and semantic violation matching with guideline-gap analytics.

2. Round 2 Submission at a Glance

Table 2. Items the Round 2 form asks for, and where this document covers them.

Required item	Where to find it
Working project link	Cover page and Section 2.1
Source-code repository link	Cover page (public GitHub repository)
Updated project PDF	This document, exported to PDF
List of completed features	Section 4, with a status for every feature
Technology and implementation details	Sections 6 to 9 and Section 11
Changes from the original plan	Section 12
Three major Round 3 improvements	Section 14

2.1 How a judge can test the core workflow

The whole path takes about five minutes. The demo accounts below all use the password 12345678. They hold invented sample data only.

Table 3. Demo accounts.

Email	Role	Use it to
saar@test.com	Creator	Write copy, run the audit, adopt suggestions, submit for review.
ss@test.com	Reviewer	Open the Review Terminal, compare V1 and V2, approve with a justification.
[Brand Manager email]	Brand Manager	Open the Governance Center and the Knowledge Base explorer.
No demo login	Agency	Role exists in the data model, with confidential facts redacted. No workspace yet (Section 15).
No demo login	Admin	Role defined only. No workspace yet (Section 15).

Steps.

1. Open https://hacksmith.vercel.app/login and sign in as saar@test.com (Creator).
2. Open Creator Studio and paste the sample caption: "Meet the world's best smart lamp! Now available across the US. Grab yours today and make the ultimate game-changer part of your home."
3. Run the audit. The scanline beam sweeps the text and the findings appear as highlights with matching cards (Section 5.1).
4. Try to submit the draft as it is. The compliance gate blocks it while critical or high-severity findings are open.
5. Adopt the suggested fixes, one click each. The text is replaced in place, re-audited and saved as Version 2. Submit the draft for review, then sign out.
6. Sign in as ss@test.com (Reviewer). Compare V1 and V2 in the Review Terminal, then approve. If warnings remain, a justification is required.
7. Sign in as the Brand Manager for the Governance Center and the Knowledge Base explorer.

3. Problem and Round 1 Recap

The problem statement treats brand governance as a coordination failure, not a writing failure [1]. Employees, agencies and AI tools publish at the same time. Approved facts go missing or out of date, terminology drifts, promotional claims lack approved evidence, and manual review leaves no record that explains a decision. Advertising standards in several jurisdictions expect a claim to be substantiated before it is published [2][3], and generative models can produce unfaithful text [6]. This is why every finding in BrandGuard AI points to approved evidence and why a human makes the final decision.

Round 1 committed us to one reliable path: submit, analyse, explain, revise, review, record. Round 2 builds that path. The workflow below is the Round 1 design. Its structure has not changed.

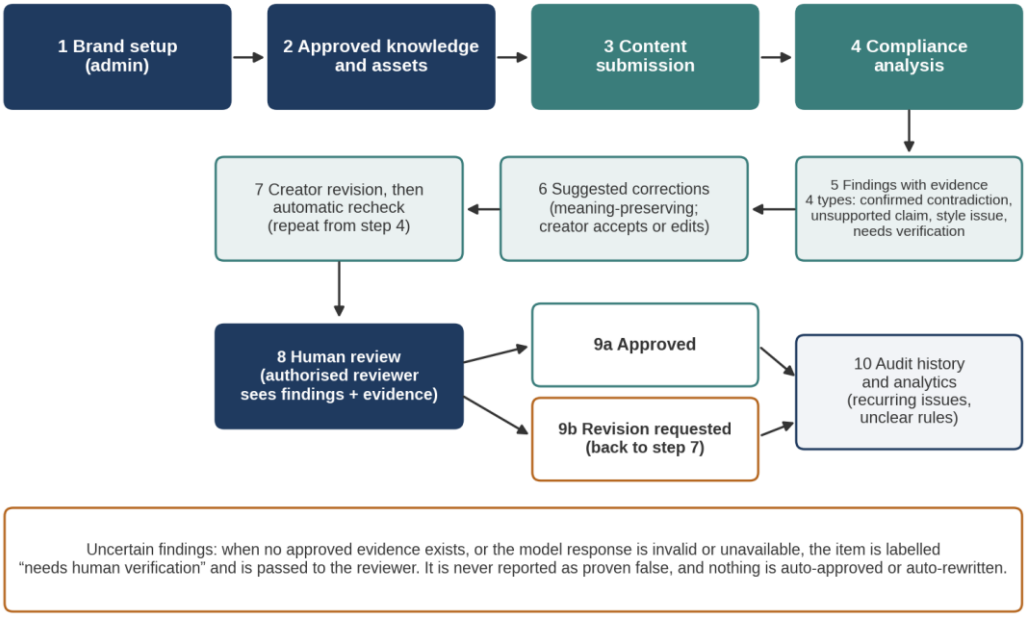


Figure 1. End-to-end workflow from the Round 1 plan, implemented in Round 2. Uncertain findings go to the reviewer and are never reported as proven false.

4. Completed Features

Status key: **Done** means a judge can use the feature on the live link. **Partial** means it works with limits stated in the Notes column. **Not done** means it is not in the submitted build.

Table 4. Feature status at submission, compared with the Round 1 MoSCoW tiers.

Feature	Round 1 tier	Status	Where to see it or notes
Role-based sign-in with isolated workspaces for Creator, Reviewer and Brand Manager	Must	Done	Login page. Each role lands in its own workspace.
Deterministic audit engine with code-point offsets (superlatives, territory contradictions, banned hype terms)	Must	Done	Creator Studio.

Feature	Round 1 tier	Status	Where to see it or notes
Highlighted editor with scanline audit beam and synchronised finding cards	Must	Done	Creator Studio.
One-click suggestion adoption with automatic re-audit and Version 2	Must	Done	Finding cards in Creator Studio.
Submission gate: open critical or high findings block submission (HTTP 422)	New in R2	Done	Submit button in Creator Studio.
Review Terminal with V1/V2 diff and decision logging	Must	Done	Reviewer workspace.
Reviewer justification required when approving with warnings	New in R2	Done	Review Terminal.
Multi-engine status bar (rule status, severity counts, LLM fallback state)	Must	Done	Creator Studio.
Knowledge Base explorer: live query of rules and approved facts	Must	Done	Explore and query. Creating or editing records from the UI is not part of this build.
Confidential fact redaction for agency accounts	New in R2	Done	Applied to API responses for agency-role users. See Section 15 on the agency workspace.
Brand Governance Center: live campaign streams, rule efficacy, dismissal rates	Should	Done	Brand Manager workspace.
Tamper-evident audit trail of creation, audit and approval events, without raw copy	Should	Done	Stored in the database. See Section 9.
Gemini 1.5 Flash advisory review with rules-only fallback	Must (if free access)	Done	The status bar shows the fallback state.
Persistent storage in Supabase PostgreSQL	Must	Done	Data survives redeploys.
Five roles enforced on the server	New in R2	Partial	Five roles are defined. Three have working workspaces.
Labelled test set and automated evaluator	New in R2	Partial	Scenarios were tested by hand (Section 10). There is no labelled set or evaluator script yet.
Image and alt-text checks	Should	Not in Round 2 scope	Planned for Round 3 (Section 14.2).
PDF and document ingestion	Could	Not in Round 2 scope	Planned for Round 3 (Section 14.1).

5. Core Workflow in the Prototype

5.1 Sample run

We use a short sample caption written to trigger one finding of each kind: a superlative, a territory contradiction and a banned hype term. The sample caption is:

"Meet the world's best smart lamp! Now available across the US. Grab yours today and make the ultimate game-changer part of your home."

Table 5. Findings expected for the sample caption. Offsets are Unicode code points with the end exclusive. We counted them by hand and checked them in Python.

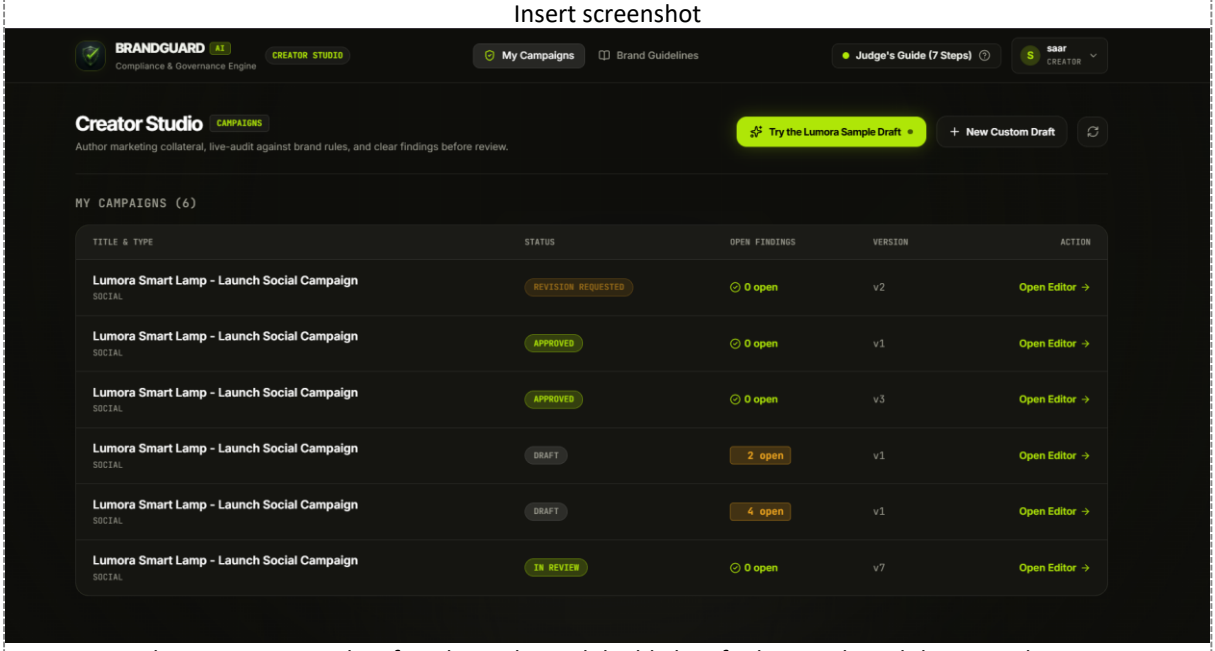
ID	Excerpt	Offsets	Type	Severity	Evidence	Shown in
F-1	world's best	9 to 21	unsupported_claim	high	Superlative rule	Figure 2
F-2	available across the US	38 to 61	confirmed_contradiction	critical	Approved territory fact	Figure 2
F-3	ultimate game-changer	93 to 114	style_issue	medium	Banned hype-term rule	Figure 2

One click on a suggestion replaces the flagged text in place. The system re-audits the new text and saves it as Version 2. When no critical or high findings remain, the draft can be submitted. The reviewer then records a decision. Only the reviewer's decision changes a submission to approved.

Note: Round 1 listed the offsets for F-1 as 11 to 23. Counting code points gives 9 to 21, and the table above uses the corrected values.

5.2 Screens

Insert screenshot



here: Creator Studio after the audit, with highlights, finding cards and the status bar

Figure 2. Creator Studio: highlighted excerpts beside synchronised finding cards, with the multi-engine status bar.

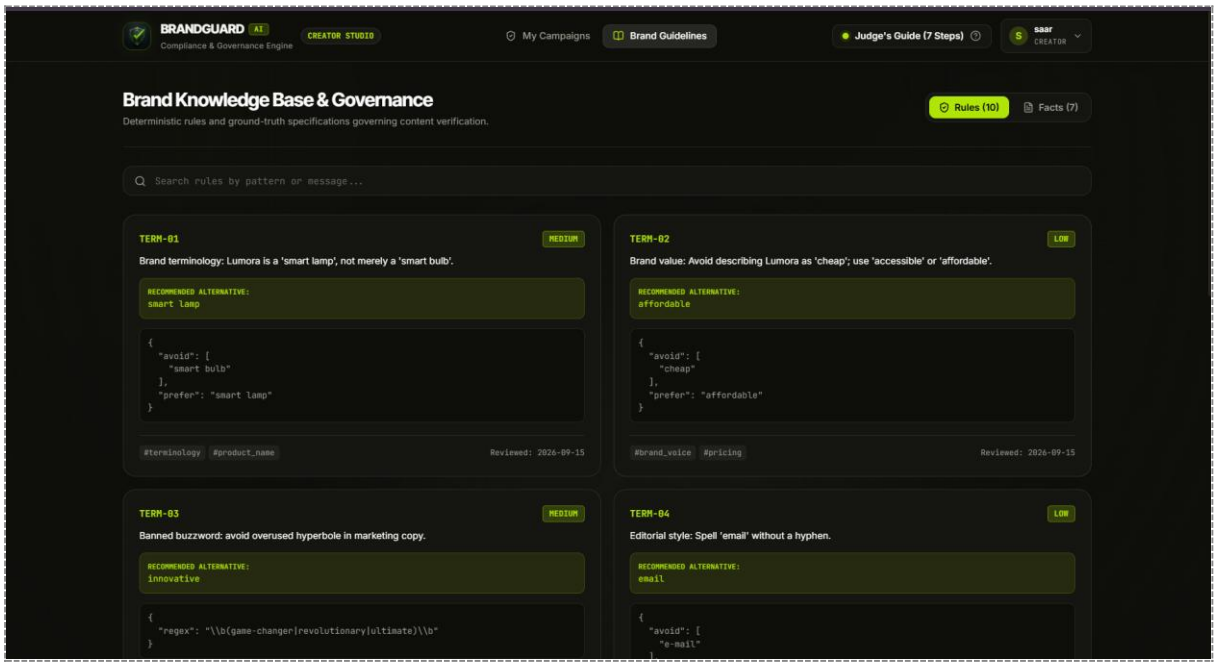


Figure 3. Knowledge Base explorer: the approved rules and facts the engine checks against.

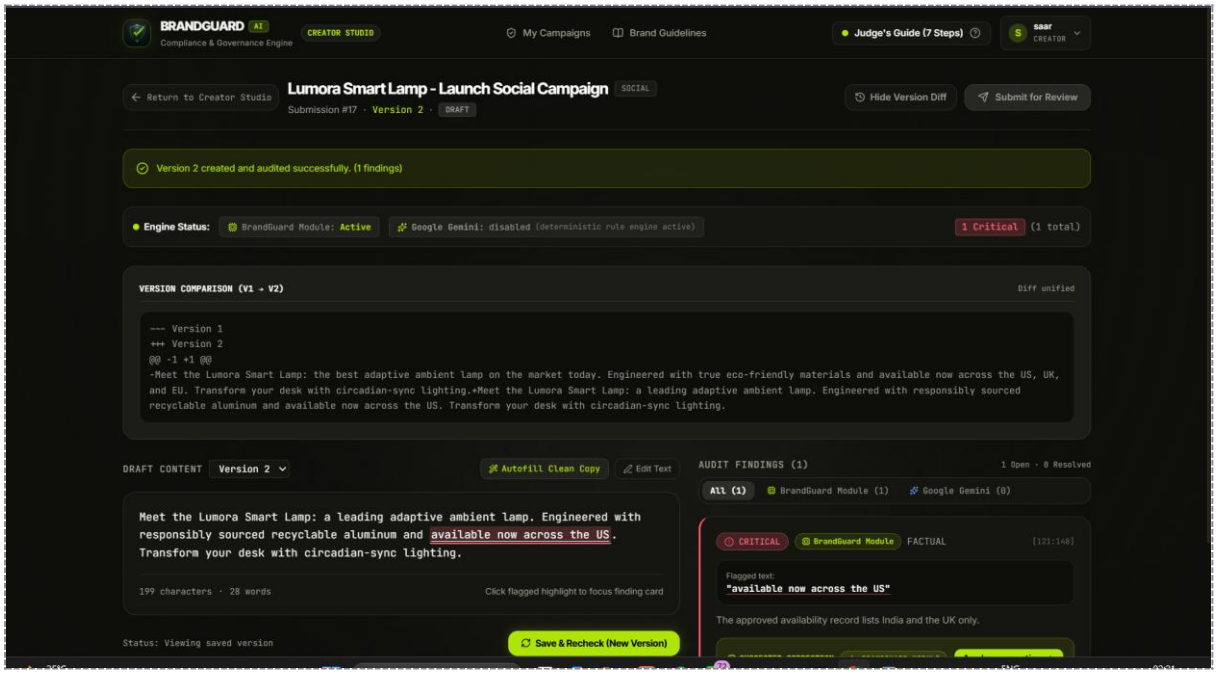


Figure 4. Review Terminal: Version 1 and Version 2 side by side, with the justification field for approving a draft that has warnings.

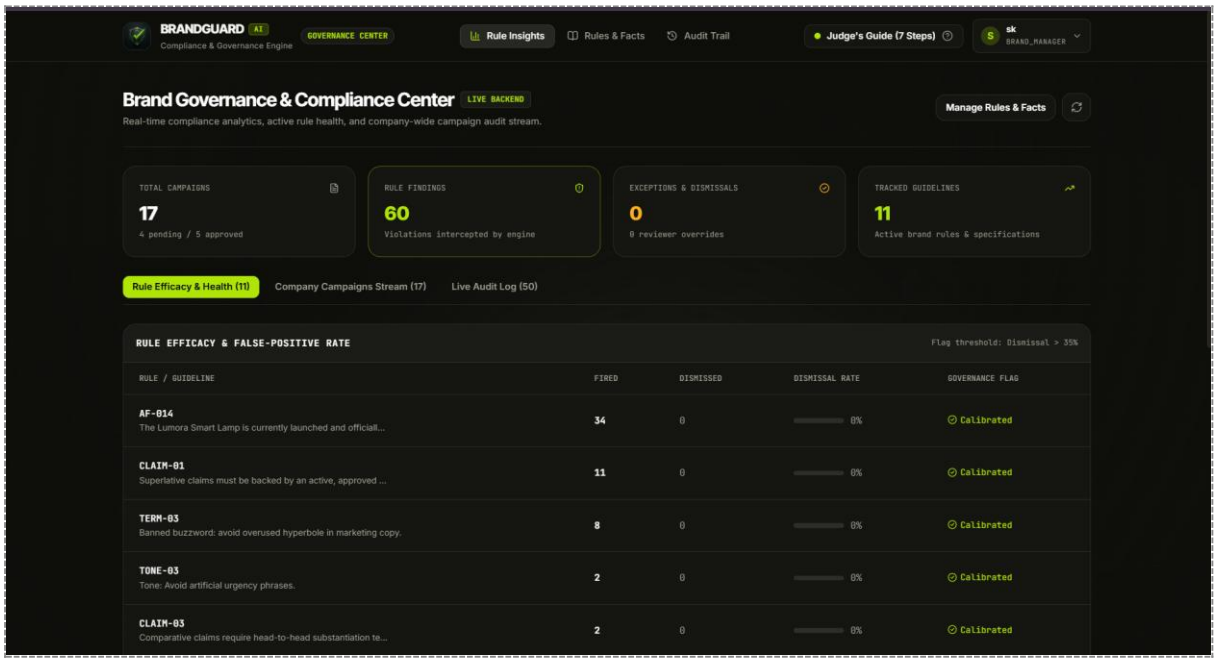


Figure 5. Brand Governance Center: live campaign streams, rule efficacy and dismissal rates.

5.3 Submission states and rules

Table 6. Submission states.

State	Meaning and how it changes
`draft`	The creator is editing and rechecking. Each save creates a new version.
`in_review`	Sent to a reviewer. The creator can no longer edit this version.
`revision_requested`	A reviewer returned it with a comment. The creator revises, which starts a new version.
`approved`	Set only by a reviewer decision. The system never approves or publishes on its own.

- A draft with an open critical or high-severity finding cannot be submitted. The server refuses it with HTTP 422, so the block does not depend on the button being disabled in the interface.
- A reviewer who approves a draft that still carries warnings must write a compliance justification. The justification is stored in the audit trail.
- Every route checks the caller's role on the server. Hiding a button in the UI is not treated as access control.

6. Architecture and Technology Stack

The build is one front end, one API service and one database, with an advisory LLM adapter. It keeps the Round 1 shape. The main change is that production data now lives in PostgreSQL.

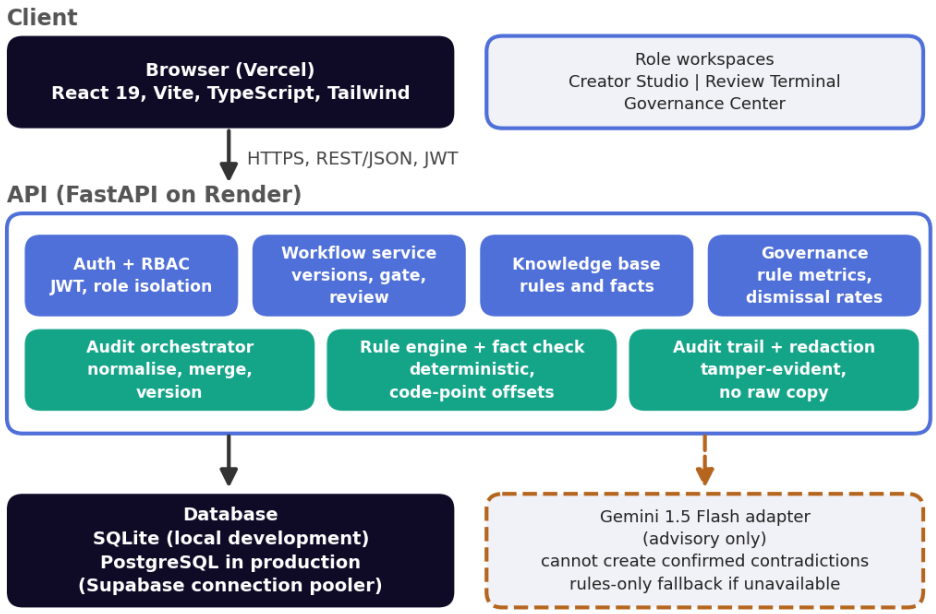


Figure 6. System architecture. The dashed path to the Gemini adapter is not on the critical path. The app works fully without it.

Data flow: the browser calls a REST endpoint with a JWT. The API layer checks the caller's role. The audit orchestrator converts the text to Unicode code points and runs the deterministic rules and approved-fact checks against the knowledge base in PostgreSQL. Findings come back with exact [start:end] offsets, and the front end draws matching highlight spans. Applying a suggestion slices the text at those offsets, which creates the next version and triggers a re-audit. In parallel, the Gemini adapter may add advisory suggestions without blocking the rule engine. The workflow service enforces the submission gate and records reviews, and every event goes to the audit trail without the raw copy. Responses to agency accounts have confidential facts redacted before they leave the API.

Table 7. Technology decisions and trade-offs.

Layer	Choice	Reason	Trade-off
Front end	React 19, TypeScript, Vite, Tailwind CSS, Lucide icons [16]	Typed components for three role-specific workspaces. Fast iteration.	More setup than plain HTML. Worth it for three separate interfaces.
Animation	Framer Motion / Motion [19]	The scanline audit beam and smooth state changes.	Extra bundle weight. Used only where it helps the audit flow.
Back end	FastAPI on Python 3.11, Pydantic v2, SQLAlchemy 2.0 [13][14][15]	Typed request and response models, automatic API docs, and clear validation errors such as the 422 gate.	Not built for very high concurrency. Fine at this scale.
Database	SQLite locally, PostgreSQL in production (Supabase pooler) [15]	Zero setup in development. Free hosts have ephemeral disks, so production needs a managed database.	Two engines to keep compatible. SQLAlchemy hides most of the differences.

Layer	Choice	Reason	Trade-off
Hosting	Vercel (front end), Render (API) [18]	Deploys from Git, HTTPS by default.	Render web services can sleep when idle, so the first request can be slow.
Rule engine	Python, Unicode normalisation, pattern and fact matching	Deterministic, explainable, testable and independent of any external service.	Misses paraphrased violations. Round 3 semantic matching addresses this.
LLM (optional)	Google Gemini 1.5 Flash [11][12]	Advisory style and tone suggestions without paid access.	Free-tier limits and terms can change. The app works without it.
Authentication	JWT (HS256) with role claims	Stateless sessions and server-side role checks.	The signing secret must stay private and be rotated if it leaks.

Table 8. Environments.

Environment	Front end	API	Database
Local development	Vite dev server	Uvicorn on localhost	SQLite file
Production	https://hacksmith.vercel.app	Render web service (see Table 1)	Supabase PostgreSQL (connection pooler)

7. Audit Engine and AI Methodology

7.1 Pipeline

1. **Normalise.** Convert the text to an array of Unicode code points, so every offset means the same thing in Python and in the browser.
2. **Deterministic rules.** Match banned hype terms, enforce preferred wording, and flag superlatives and absolutes that need substantiation.
3. **Fact lookup.** Compare checkable statements, such as the territories where a product is sold, with the approved fact records. A conflict becomes a `confirmed_contradiction`.
4. **Retrieve context.** Select the small set of rules and facts relevant to the draft, by tag and keyword.
5. **Optional Gemini review.** Ask Gemini 1.5 Flash for tone and messaging suggestions that exact matching misses. This runs asynchronously, so it never blocks the rule engine.
6. **Validate.** Check the model output against a schema. Keep a suggestion only if its type is allowed, and never let it override a rule result.
7. **Merge and store.** Combine rule and model findings, attach the offsets, and save the result with the version. Log the event without the raw copy.

7.2 Finding types

Table 9. The four finding types and who may produce them.

Type	Meaning	Produced by	Reviewer action
`confirmed_contradiction`	The text conflicts with a current approved fact or rule.	Rules and facts only	Check the record is current. Require a fix.

Type	Meaning	Produced by	Reviewer action
`unsupported_claim`	An objective or superlative claim with no approved substantiation. Not declared false.	Rules or LLM	Ask for evidence, add it to the knowledge base, or remove the claim.
`style_issue`	Prohibited phrase, unapproved term or tone departure.	Rules or LLM	Accept an exception or request a change.
`needs_human_verification`	No relevant evidence, ambiguous rules or an unusable model response.	Rules or LLM	Decide, and optionally add a rule or fact.

7.3 Offsets and highlighting

Offsets are Python Unicode code points, with the end exclusive. JavaScript strings use UTF-16, so slicing by index splits surrogate pairs and shifts every highlight that follows an emoji or any other character outside the Basic Multilingual Plane. The front end therefore converts the text with `Array.from()` and slices by code points. This keeps the highlight spans in the browser in one-to-one agreement with the offsets Python returns. Scenario T9 in Section 10 covers this.

7.4 What the model is and is not trusted to do

Gemini is a second reviewer, not an authority. It gives advisory suggestions only, it has no tools or write access, and the deterministic engine alone can raise a confirmed contradiction. We show no numeric confidence score, because a language model's number is not known to be calibrated.

Table 10. Failure modes and controls.

Risk	Why it matters	Control in the prototype
Hallucinated findings	Models can produce text that is not grounded in their input [6].	Only the rule engine and approved facts can create a confirmed contradiction. Model output stays advisory.
Prompt injection through the draft	Instructions inside content can change model behaviour [7][8].	The model has no tools or write access, and rule results never depend on it.
Outdated fact	A stale record causes a false contradiction.	The Knowledge Base explorer shows exactly what the engine checks against, so a brand manager can spot a stale record.
LLM unavailable or rate-limited	Free tiers have no guaranteed limits [11].	The audit falls back to rules only, and the status bar shows the fallback state.
False positives	Over-flagging erodes trust.	The Governance Center tracks dismissal rates per rule, so a brand manager can fix rules that are dismissed often.

LLM configuration. Google Gemini 1.5 Flash through a backend adapter. It is optional and advisory. Free-tier data terms apply [12]. Only invented demo data is sent.

8. Data Model and API Contract

The schema follows the Round 1 proposal with the Round 2 changes. The exact definitions are in the SQLAlchemy models in the repository.

Table 11. Database tables.

Table	Main fields
users	id, name, email, role (creator, agency, reviewer, brand_manager, admin)
rules	id, type (term, phrase, claim, tone), pattern, severity, message, suggestion, status
facts	id, topic, statement, structured_value, valid_from, last_reviewed, status
submissions	id, title, content_type, author_id, status (draft, in_review, revision_requested, approved)
versions	id, submission_id, number, body, created_at
findings	id, version_id, category, severity, start, end, explanation, evidence_id, suggestion, verification_status, source_engine, resolution (open, accepted, edited, dismissed), dismissal reason
reviews	id, submission_id, reviewer_id, decision, comment, override_reason, created_at
audit_log	id, actor_id, action, object_id, created_at. Tamper-evident. Stores no raw copy.

Table 12. Main endpoints, as designed. The full, current list is on the API's /docs page (see Table 1).

Endpoint	Purpose	Roles
POST /api/submissions	Create a submission and Version 1.	creator, agency
POST /api/submissions/{id}/audit	Run the audit on the latest version. Returns findings and which engines ran.	creator, agency
POST /api/submissions/{id}/versions	Save a revised body (for example after a one-click fix), then re-audit.	creator, agency
PATCH /api/findings/{id}	Accept, edit or dismiss a finding.	creator, agency
POST /api/submissions/{id}/submit	Send for review. Returns HTTP 422 while critical or high findings are open.	creator, agency
POST /api/submissions/{id}/decision	Record a decision with a comment. A justification is required when warnings remain.	reviewer
GET /api/submissions/{id}	Submission, versions, findings and reviews.	Role-limited
GET /api/knowledge/{rules|facts}	Query rules and approved facts. Confidential facts are redacted for agency accounts.	creator, reviewer, brand_manager, agency (redacted)
GET /api/insights/recurring	Rule counts and dismissal rates for the Governance Center.	brand_manager
GET /health	Liveness check.	Public
Authentication routes	Sign in and receive a JWT (HS256).	Public

9. Security, Privacy and Responsible Design

The problem statement requires a balance of automation and human control, protection of confidential brand material, and disclosure of AI use [1]. We use the NIST AI Risk Management Framework [10] as a

checklist for our own thinking. We do not claim conformity with it, legal compliance, certification or a security audit.

Table 13. Controls in the prototype.

Area	Control	Status
Role-based access	Roles are enforced on the server. Creator, Reviewer and Brand Manager each get an isolated workspace.	Done
Authentication	Sign-in issues a JWT signed with HS256. The role travels with the token and is checked on every request.	Done
Human control	Only a reviewer decision approves content. Approving a draft with warnings needs a written justification.	Done
Compliance gate	Open critical or high findings block submission with HTTP 422.	Done
Audit trail	Creation, audit and approval events are logged in a tamper-evident trail. Raw copy is never stored in it.	Done
Prompt injection	Gemini has no tools or write access, and its output is advisory only.	Done
LLM scope	The LLM cannot raise a confirmed contradiction. Rules-only mode is always available.	Done
Third-party AI disclosure	The status bar shows whether the LLM review ran or fell back. Only invented demo data is sent.	Done
Secrets	API keys and database URLs are environment variables on Render and Vercel.	Done
Transport	HTTPS through the Vercel and Render defaults.	Done
Demo credentials	Shared demo password on invented accounts only. We will change or remove the accounts after judging.	Planned
Confidential facts	Responses to agency accounts have confidential facts redacted, so contractors can audit copy without seeing internal statements.	Done

10. Testing and Evaluation Results

We tested the live build by running the scenarios in Table 14 by hand. We did not build a labelled test set or compute precision and recall in Round 2, so we report no accuracy figures. The same team wrote the rules and the scenarios, so a pass here shows the workflow behaves as designed. It does not show that the engine generalises to other brand guidelines.

Table 14. Test scenarios and results.

ID	Scenario	Expected behaviour	Result
T1	Clean copy with no issues	No findings. The draft can be submitted.	Pass
T2	Banned hype term	Finding on the exact words, with a suggested replacement.	Pass
T3	Unsupported superlative	High-severity finding on the superlative.	Pass
T4	Territory claim that contradicts an approved fact	Critical finding on the territory claim.	Pass
T5	One-click suggestion	Text replaced in place, re-audited, Version 2 created.	Pass

ID	Scenario	Expected behaviour	Result
T6	Submit with an open critical or high finding	Refused by the server with HTTP 422. The draft stays with the creator.	Pass
T7	Submit a clean draft	The draft appears in the Review Terminal with the V1 and V2 diff.	Pass
T8	Reviewer approves a draft with warnings	Blocked until a justification is entered. Decision and justification are logged.	Pass
T9	Copy containing emoji	Highlights stay aligned with the correct text.	Pass
T10	Agency account views a confidential fact	The fact is redacted in the response.	Pass
T11	Gemini unavailable	The status bar shows the fallback state. Rule findings are still returned.	Pass
T12	Audit trail after a full run	Entries for creation, audit and approval. No raw copy stored.	Pass
T13	Creator tries a reviewer-only action	Refused by the server.	Pass

What we have not measured. Precision, recall and audit latency were not measured in Round 2. Round 3 adds a labelled test set, which Improvement 3 needs anyway. We will report those figures then, with the same caveat about rules written by the team.

11. Deployment and Reliability

11.1 Release process

The code lives in one public GitHub repository. Vercel serves the front end and Render runs the API, both deployed from the repository. Production data is in Supabase PostgreSQL through the connection pooler. The demo knowledge base and demo accounts are seeded into the database. Work was merged from feature branches (Section 13) into the main branch before each deployment.

11.2 Failure handling

Table 16. Dependency failures and what the user sees.

Failure	System behaviour	What the user sees
LLM unavailable	Skip the Gemini review and run rules only.	The status bar shows the fallback state.
LLM rate limit	Same fallback. The rule engine is unaffected.	The same status bar notice.
No relevant approved evidence	Do not call the claim false. Flag it for the reviewer.	A flagged item for human review.
Malformed model response	Discard output that fails validation. Keep the rule findings.	Rule findings only.
API cold start	The Render service may need a few seconds to wake after idle time.	The first request can be slow. Wait a few seconds and try again.
Database write failure	The request fails and nothing partial is saved.	An error message. The text stays in the editor.

12. Changes from the Round 1 Plan

Table 17. What changed from the Round 1 proposal, and why.

Area	Round 1 plan	Round 2 build	Reason
Database	SQLite, with PostgreSQL as a later upgrade.	SQLite locally, PostgreSQL in production (Supabase pooler).	Free host disks are ephemeral, so a SQLite file would lose data on redeploy. SQLAlchemy 2.0 keeps the code the same in both.
Hosting	Local demonstration.	Vercel (front end) and Render (API).	Judges must be able to open a live link.
Roles	Creator, marketing manager, brand manager, approver, agency, admin, with separate legal and marketing review screens.	Five roles in the data model. Working workspaces for Creator, Reviewer and Brand Manager. Legal and marketing review merged into one Compliance Reviewer.	The two review screens overlapped. One reviewer role with a mandatory justification is simpler to enforce on the server. We did not have time to build agency and admin workspaces.
Workflow gates	Advisory warnings only. The creator decided what to do.	A hard gate: open critical or high findings block submission with HTTP 422.	A critical contradiction should not reach a reviewer as if it were acceptable.
Confidentiality	A limited view of internal facts for agencies (planned).	Agency accounts receive responses with confidential facts redacted.	Contractors can check copy without seeing confidential trade statements.

Area	Round 1 plan	Round 2 build	Reason
LLM scope	The model may flag tone, messaging and claim issues.	Gemini is advisory only. Only the rule engine can raise a confirmed contradiction.	Only a stored approved fact can establish a contradiction.
Offsets	Character offsets.	Python Unicode code points, with an Array.from() normaliser in the browser.	JavaScript UTF-16 indexes break highlights after emoji. The Round 1 sample offsets for F-1 were also wrong and are corrected.
Audit trail	A log of every action.	A tamper-evident audit trail that stores no raw copy.	Reviewers need a record they can trust, and the log should not leak draft copy.
Interface	Tailwind styling, no behaviour defined.	Scanline audit beam, synchronised highlight and card views, and a status bar.	Clearer feedback while the audit runs, and a polished first impression.

13. Team Contributions and Execution

Table 18. Branch ownership.

Member	Branch	Scope
Sagar Kharbikar	`sagar/backend-core`	Database, authentication, workflow, review, knowledge CRUD, seed data, CI and deployment.
Sagar Kharbikar	`sagar/audit-engine`	Rules, facts, evidence resolver, orchestrator, insights, test set and evaluator.
Ayush Roy	`ayush/frontend-workspace`	App shell, login, creator workspace, highlighted editor, findings.
Ayush Roy	`ayush/review-ai-insights`	Reviewer UI, knowledge editor, insights, audit log view, LLM adapter in `backend/app/llm/`.

Sagar is also the integrator for deployment and the submission form. Both members share the API contract, bug triage and the demo script.

14. Three Major Round 3 Improvements

Each improvement adds a capability that Round 2 does not have. None is a design or cosmetic change. All three build on the shipped code: the deterministic engine, the four finding types and the review workflow stay as they are.

14.1 Automated guideline ingestion from documents

What. A brand manager uploads raw brand-guideline PDFs, product spec sheets and legal decks. Layout-aware OCR and structured parsing turn them into proposed deterministic rules and verified facts, each with a citation to its source page. Nothing becomes active until a brand manager approves it.

Why it matters. Today the knowledge base is seeded by hand, which means hours of writing patterns for every new client or product launch. Ingestion removes the main adoption barrier and gives every imported rule a traceable source.

How. Text and layout extraction with page tracking, OCR for scanned pages and slide images, an extraction step that proposes rules and facts in the existing rule and fact schema, an approval queue for the brand manager, and strict file-type and size limits with untrusted-content handling.

Done when. A sample guideline PDF of about 10 pages yields candidate rules and facts that each cite their source page. A candidate has no effect on any audit until a brand manager approves it. A scanned page is handled through OCR.

Risk and fallback. Extraction quality varies with layout. If a file cannot be parsed reliably, the system shows the raw extracted text for manual rule entry instead of guessing.

14.2 Visual and accessibility compliance engine

What. Review of uploaded marketing creatives and social banners, not only text. The checks cover conformance to the brand colour palette, text-on-image contrast against WCAG 2.2 [5], logo safe-zone padding, and the presence of mandatory alt text. Creators and reviewers see bounding-box overlays on the exact area that fails.

Why it matters. The problem statement names visual consistency and accessibility, and Round 1 deferred them as the hardest part to automate [1][4]. This brings them into the same evidence-backed workflow as text.

How. Image upload with allow-listed types, palette distance and WCAG contrast calculations in Python, OCR to locate text regions, template matching against the registered logo to measure safe-zone padding, and findings that carry a bounding box so the front end can draw the overlay. Brand colours, the logo and alt-text rules are stored as asset records in the knowledge base.

Done when. An image with an off-palette colour, a low-contrast text region, a logo too close to the edge, or missing alt text produces a finding with a bounding box. A compliant image produces none.

Risk and fallback. Logo detection is unreliable on busy images. The first release limits matching to the registered logo asset, and anything uncertain is labelled `needs_human_verification`.

14.3 Semantic violation matching and guideline-gap analytics

What. Dense vector embeddings stored with pgvector sit beside the deterministic engine and catch paraphrased, implied or tonally off violations that exact matching misses, for example "number one" where a rule bans "best". These matches are labelled `needs_human_verification` unless a rule also

fires. A dispute-clustering engine in the Brand Manager dashboard flags rules that are often overridden or dismissed, and proposes clearer guideline wording for the brand manager to approve.

Why it matters. The rule engine's main known gap is paraphrase. The Governance Center already shows dismissal rates, but it does not yet help fix the rules behind them.

How. pgvector [17] on the PostgreSQL database we already run in production, with embeddings computed when a rule or fact is created. Clustering groups similar dismissal and override reasons per rule. Proposed wording is a suggestion and is never applied automatically. We will confirm pgvector support on our Supabase plan before starting.

Done when. On a labelled paraphrase set, retrieval finds the relevant rule for most paraphrases that exact matching misses, and the false-positive rate on clean text does not rise. The dashboard lists the most overridden rules with a proposed rewording for each. We will report the measured numbers with the same caveat as Section 10.

Risk and fallback. Embedding calls may hit free-tier limits. Rules and facts are few, so embeddings can be precomputed once. If retrieval is unavailable, the audit falls back to the current engine.

14.4 Considered and not chosen

Publishing-platform integrations and multi-tenant organisation isolation are valuable, but they add scope without improving the review quality that judges and users see. They stay on the longer roadmap.

15. Limitations, Risks and Known Issues

- The knowledge base is small and written by the team, so results will not carry over to real guideline documents, which are often inconsistent.
- We have no labelled test set. The scenarios in Section 10 were run by hand by the team. They show that the workflow works, not how accurate the engine is.
- Free-tier LLM access can change or disappear without notice [11]. The app is designed to work without it.
- The build reviews text only. Image and document review are Round 3 work.
- Render web services can be slow on the first request after idle time.
- No user interviews or feedback sessions have been run, so the personas and workflow remain assumptions.

Table 19. Known issues at submission.

ID	Issue	Impact	Workaround
K1	Five roles are defined in the data model, but only Creator, Reviewer and Brand Manager have working workspaces.	Agency and admin users cannot do their own tasks in the app.	Use the three working accounts in Table 3. Agency and admin workspaces are future work and are not one of the three Round 3 improvements.
K2	The Knowledge Base explorer is read and query only.	Rules and facts cannot be created or edited from the interface.	Records are seeded into the database. Round 3 ingestion adds an approval queue.
K3	The Gemini review depends on a free-tier model.	When the call fails, only rule-based findings are shown.	The status bar shows the fallback state. The rule engine is complete on its own.

16. Conclusion

BrandGuard AI now runs the workflow it proposed in Round 1. A creator audits copy, a deterministic engine returns findings with exact offsets, a gate keeps serious violations out of review, and a human reviewer decides with both versions in view. Round 2 added the parts a live product needs: PostgreSQL storage, role enforcement, a tamper-evident audit trail and a deployed link. Whether it delivers the benefits we expect still has to be shown by wider testing. Round 3 targets the largest gaps: guideline documents, visual compliance and paraphrase detection.

References

Numbers match the citations in the text. Numbers [1] to [12] are unchanged from the Round 1 document.

[1] Inkloom AI. Problem Statement by Inkloom AI (including round submission requirements, judging criteria and general rules). Official event document supplied to participants as "Problem_statement.pdf". No public URL was provided.

[2] U.S. Federal Trade Commission. FTC Policy Statement Regarding Advertising Substantiation (appended to Thompson Medical Co., 104 F.T.C. 648, 1984). https://www.ftc.gov/sites/default/files/attachments/training-materials/policy_substantiation.pdf

[3] Advertising Standards Council of India (ASCI). The Code for Self-Regulation of Advertising Content in India. https://www.ascionline.in/the-asci-code

[4] WebAIM. The WebAIM Million: The 2025 report on the accessibility of the top 1,000,000 home pages. https://webaim.org/projects/million/2025

[5] W3C. Web Content Accessibility Guidelines (WCAG) 2.2, W3C Recommendation, October 2023. https://www.w3.org/TR/WCAG22/

[6] Ji, Z., et al. "Survey of Hallucination in Natural Language Generation." ACM Computing Surveys, 55(12), 2023. https://doi.org/10.1145/3571730

[7] OWASP GenAI Security Project. LLM01:2025 Prompt Injection. https://genai.owasp.org/llmrisk/llm01-prompt-injection/

[8] Greshake, K., et al. "Not what you've signed up for: Compromising Real-World LLM-Integrated Applications with Indirect Prompt Injection." AISec '23, 2023. https://arxiv.org/abs/2302.12173

[9] Lewis, P., et al. "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks." NeurIPS 2020. https://arxiv.org/abs/2005.11401

[10] National Institute of Standards and Technology. Artificial Intelligence Risk Management Framework (AI RMF 1.0), NIST AI 100-1, 2023. https://doi.org/10.6028/NIST.AI.100-1

[11] Google. Gemini API: Rate limits. https://ai.google.dev/gemini-api/docs/rate-limits

[12] Google. Gemini API: Billing and free/paid tiers. https://ai.google.dev/gemini-api/docs/billing

[13] FastAPI documentation. https://fastapi.tiangolo.com/

[14] Pydantic documentation. https://docs.pydantic.dev/latest/

[15] SQLAlchemy documentation. https://docs.sqlalchemy.org/ ; SQLite documentation, https://www.sqlite.org/docs.html ; PostgreSQL documentation, https://www.postgresql.org/docs/

[16] React documentation. https://react.dev/ ; Vite, https://vite.dev/guide/ ; Tailwind CSS, https://tailwindcss.com/docs

[17] pgvector: open-source vector similarity search for PostgreSQL. https://github.com/pgvector/pgvector

[18] Vercel documentation. https://vercel.com/docs ; Render documentation, https://render.com/docs

[19] Motion (animation library for React) documentation. https://motion.dev/docs

Appendix A. Sample Knowledge Base and Audit Response

Invented demo data. Not a real customer. The excerpt below shows the shape of the knowledge base records and the audit response. It is illustrative and is not copied from the live API.

```
// knowledge base (excerpt)
{"id":"AF-014","type":"fact","topic":"availability",
 "statement":"Lumora Smart Lamp is currently sold in India and the United Kingdom only",
 "structured_value":{"regions":["IN","GB"]},"status":"current"}
{"id":"TERM-03","type":"phrase","pattern":["game-changer","revolutionary","ultimate"],
 "severity":"medium","suggestion":"Use an approved descriptor such as TAG-01"}
{"id":"CLAIM-01","type":"claim","pattern":["best","number one","#1"],
 "severity":"high","message":"Superlatives need an approved substantiation record"}

// POST /api/submissions/{id}/audit  ->  200 (illustrative)
{"version":1,"engines":{"rules":"ran","llm":"unavailable"},
 "findings":[
  {"issue_id":"F-1","category":"claim","severity":"high",
   "excerpt":{"text":"world's best","start":9,"end":21},
   "evidence":{"id":"CLAIM-01","kind":"rule"},
   "verification_status":"unsupported_claim","source_engine":"rules"},
  {"issue_id":"F-2","category":"factual","severity":"critical",
   "excerpt":{"text":"available across the US","start":38,"end":61},
   "evidence":{"id":"AF-014","kind":"fact"},
   "suggested_correction":"available in India and the UK",
   "verification_status":"confirmed_contradiction","source_engine":"rules"},
  {"issue_id":"F-3","category":"terminology","severity":"medium",
   "excerpt":{"text":"ultimate game-changer","start":93,"end":114},
   "evidence":{"id":"TERM-03","kind":"rule"},
   "verification_status":"style_issue","source_engine":"rules"}]}
```

Appendix B. Alignment with Round 2 Judging Criteria

Table 20. Round 2 criteria (100 points) mapped to sections.

Criterion	Points	Where addressed	Main content
Working core workflow	30	Sections 2.1, 4, 5	Judge walkthrough, feature status, sample run and screens.
Technical depth and reliability	25	Sections 6 to 11	Architecture, audit engine, LLM guardrails, data model, security, tests, deployment and failure handling.
Progress from Round 1	20	Sections 3, 4, 12	Round 1 recap, feature status against MoSCoW tiers, changes and reasons.
Three planned improvements	15	Section 14	Ingestion, visual and accessibility review, semantic retrieval with guideline-gap analytics, each with acceptance criteria.
Usability	10	Sections 2.1, 5	Demo accounts, step-by-step path, role-specific views, clear failure messages.

Tools and AI Disclosure

The official rules require disclosure of significant AI assistance, reused material, templates and external services [1].

Table 21. Tools, services and AI assistance.

Item	Use and disclosure
Claude (Anthropic)	Used for planning, drafting documentation, and pair-programming implementation across the FastAPI backend routes, character-offset normalization logic, and React frontend UI components. The engineering team reviewed, debugged, tested, and validated all generated code and architecture.
LLM in the product	Google Gemini 1.5 Flash. Used strictly in an optional, advisory capacity for tone and style analysis. Operates with zero authority over deterministic contradiction checks. Only invented demo data is processed.
Templates and boilerplate	None. Built custom from scratch using standard create-vite and minimal FastAPI directory structures. No third-party UI templates, commercial component kits, or pre-built themes were used.
Open-source libraries	React, Vite, Tailwind CSS, Framer Motion, Lucide, FastAPI, Pydantic, SQLAlchemy and their dependencies. See package.json and requirements.txt in the repository.
Hosting and services	Vercel, Render, Supabase and GitHub.
Data	100% invented synthetic data. Includes fictional Lumora Smart Lamp marketing captions, mock brand guideline rules, and synthetic approved factual specifications (data/seed/).